



Git & GitHub

Complete Notes

From First Commit to CI/CD — Beginner to Advanced, All in One Place

14

Chapters

80+

Concepts Covered

110+

Commands Explained

A-Z

Basics → Advanced

Flip the page for the full chapter-by-chapter breakdown

by Vaibhav Singh

Table of Contents

Fourteen chapters, laid out beginner → advanced. Jump in anywhere, or read straight through.

- **Foundations** — what version control actually is, why Git won, and how it differs from GitHub.
- **Core workflow** — the working directory → staging → commit model, branching, and merging.
- **Remote collaboration** — clone, push, pull, fetch, upstream, and how teams stay in sync.
- **Intermediate & advanced Git** — stash, tags, cherry-pick, rebase, bisect, hooks, worktrees, LFS.
- **Git internals** — the object model, SHA hashing, references, and plumbing commands.
- **GitHub the platform** — Issues, PRs, Actions (CI/CD), security tooling, and the API/CLI.
- **Real-world workflows & best practices** — Git Flow, GitHub Flow, trunk-based development.
- **Practice** — hands-on tasks, mini-projects, common mistakes, and a full command cheat sheet.

01

Introduction to Version Control

What a VCS is, and why Git won

02

Git Basics

Install, configure, add, commit, log

03

Branching & Merging

Branches, merges, and conflicts

04

Remote Repositories

Push, pull, fetch, upstream

05

Intermediate Git

Stash, tags, cherry-pick, revert

06

Advanced Git

Rebase, bisect, hooks, LFS

07

Git Internals

Objects, SHA hashing, refs

08

GitHub Collaboration

Issues, PRs, forks, releases

09

GitHub Actions (CI/CD)

Workflows, jobs, secrets

10

GitHub Security

CodeQL, Dependabot, rulesets

11

GitHub API & Automation

REST, GraphQL, gh CLI

12

Real-World Workflows

Git Flow, GitHub Flow, trunk-based

13

Best Practices

Commits, naming, clean history

14

Hands-On & Cheat Sheet

Practice, mistakes, full command list

Tip: each chapter opens with a one-line summary of exactly what it covers, so you always know where you are.

Chapter 1 — Introduction to Version Control

Before touching a single command, understand the problem Git solves — everything else in this guide builds on it.

What is Version Control?

A **Version Control System (VCS)** is software that tracks every change made to a set of files over time. Instead of overwriting a file each time you save it, a VCS keeps a full history of every version — who changed what, when, and why — and lets you jump back to any earlier state whenever you need to.

Before version control became standard, teams managed this manually: emailing files back and forth, or keeping folders named `project_final.zip`, `project_final_v2.zip`, `project_final_v2_ACTUAL_FINAL.zip`. That approach breaks down almost immediately once more than one person touches the code — nobody can tell which copy is current, changes silently overwrite each other, and there's no way to see *why* something changed. Version control replaces all of that with one authoritative, traceable history.

Real-World Use: Every professional codebase — from a solo side project to a company with thousands of engineers — relies on a VCS as its single source of truth for what the code looked like at any point in time.

Fun Fact

Before Git, developers relied on tools like **CVS** and **SVN**, where a single central server was often a single point of failure — if it went down, so did everyone's ability to commit. Linus Torvalds famously listed "no single point of failure" as one of his hard requirements when designing Git in 2005, which is exactly why every clone carries the entire project history rather than just the latest snapshot.

Centralized vs Distributed Version Control

Version control systems fall into two broad families, and the difference explains almost everything about how Git feels to use compared to older tools like SVN or CVS.

Centralized VCS (e.g. SVN)	Distributed VCS (Git)
One central server holds the full history; developers only have the current checkout	Every developer clones the <i>entire</i> history to their own machine
Needs a network connection for almost every operation — committing, viewing history, branching	Committing, branching, and browsing history all work completely offline; only push/pull need network access
If the central server goes down or is lost, the project's history is at serious risk	Every clone is effectively a full backup of the project's history
Branching is heavyweight and often discouraged	Branching is near-instant and cheap, so developers create them freely

In interviews, the single word that captures Git's edge over older systems is **"distributed"** — lead with it whenever you're asked to describe Git.

Why Git?

- Free, open-source, and extremely fast, even on huge codebases
- Works fully offline — commit, branch, and inspect history without internet
- Cheap, instant branching encourages experimentation without risk
- Became the de facto industry standard — nearly every company uses it
- Strong data integrity: every object is checksummed, so silent corruption is essentially impossible

Why GitHub?

- Provides free, reliable cloud hosting for Git repositories
- Pull Requests turn merging into a structured, reviewable conversation
- Issues and Projects give teams a place to plan and track work alongside the code
- Actions brings automated testing and deployment directly into the repo
- It's the default meeting place for open-source collaboration worldwide

Git vs GitHub — The Core Distinction

Git is a command-line *tool* that runs entirely on your machine and manages version history — it has no concept of "the cloud" built into it at all. **GitHub** is a separate, web-based *platform* that hosts Git repositories and layers collaboration features on top: pull requests, issue tracking, CI/CD, project boards, and more.

You can use Git without ever touching GitHub — plenty of solo developers and private company networks do exactly that, sometimes with alternative hosts like GitLab or Bitbucket, or no remote host at all. But GitHub cannot exist without Git; it's built as a layer on top of it.

Real-World Use: A solo developer might use Git alone for personal projects, committing locally with no remote at all. The moment a second person needs to collaborate, a remote host like GitHub becomes essential for sharing that history.

Fun Fact

Git was created in 2005 by **Linus Torvalds** — the same person behind the Linux kernel — after the community lost free access to the proprietary tool it had been using. He reportedly built the first working version in about **ten days**. Torvalds has joked that he's bad at naming things, and that "git" is British slang for an unpleasant person — his own explanation being "I'm an egotistical bastard, and I name all my projects after myself."

Despite being written in just over a week, Git has gone on to become the version control system behind the vast majority of software projects on Earth, including massive codebases at Google, Microsoft, and Meta — and, fittingly, the Linux kernel itself, the project it was originally built to manage.

Quick Tip: If you only remember one command from this whole guide, make it `git status` — running it constantly, especially when you're unsure what state your repo is in, prevents the vast majority of beginner mistakes before they happen.

Chapter 2 — Git Basics

The everyday foundation: install, configure, and understand the three-zone model that every Git command operates on.

Installation & Configuration

After installing Git, the first thing every setup needs is identity configuration. Git stamps the author name and email onto *every single commit* you make, permanently — so this is worth getting right before your first commit, not after.

```
# Set your identity (required before committing)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Set your default editor and default branch name
git config --global core.editor "code --wait"
git config --global init.defaultBranch main

# View all current settings, or just one
git config --list
git config user.email
```

Configuration in Git works at three levels, each overriding the one before it: **system** (all users on the machine), **global** (all repos for your user, via `--global`), and **local** (a single repository, the default when no flag is passed).

Pro Tip: Use `--global` for your usual identity, but override it locally in a repo where you need a different email — e.g. a work email for a company repo.

Repository, Working Directory, Staging Area & Commit

Git's entire day-to-day workflow revolves around three zones. Understanding this model is arguably the single most useful mental model in all of Git — once it clicks, almost every command's behavior becomes predictable instead of magical.

- **Working Directory** — the actual files on your disk that you open, edit, and save in your editor. This is just your normal filesystem.
- **Staging Area (a.k.a. the Index)** — an intermediate holding zone where you deliberately build up the exact snapshot you want for your next commit, using `git add`. Nothing reaches history until it's staged.
- **Repository (the .git folder)** — the permanent, committed history. Once you run `git commit`, that snapshot is saved forever in the repository (unless deliberately rewritten later).

```
Working Directory  --git add-->  Staging Area  --git commit-->  Repository (History)
(edit files)                (index)                (.git objects)
```

Real-World Use: Staging lets you commit only *part* of your changes — for example, stage and commit just the bug fix, leaving unrelated work-in-progress edits sitting uncommitted in the working directory for later.

.gitignore

A `.gitignore` file tells Git which files and folders to never track, even if they exist in the working directory. This typically includes build output, installed dependencies, environment secrets, and editor/IDE settings — things that are either regenerable or shouldn't be shared.

```
node_modules/
dist/
.env
*.log
.vscode/
__pycache__/
```

Patterns support wildcards (`*.log`), directory matches (`node_modules/`), and negation (`!important.log` to force-include one file inside an otherwise ignored pattern).

Common Mistake: Adding a file to `.gitignore` *after* it's already been committed won't untrack it — Git is already watching it. You must also run `git rm --cached <file>` to remove it from tracking while keeping it on disk.

Chapter 2 (continued) — Everyday Commands

The Core Command Set

These seven commands make up roughly 90% of daily Git usage. Everything covered later in this guide builds on top of this foundation.

Command	What it does
<code>git init</code>	Creates a new, empty Git repository in the current folder (adds a hidden <code>.git</code> directory)
<code>git clone <url></code>	Copies an existing remote repository, including its full history, to your machine
<code>git status</code>	Shows which files are staged, unstaged, or untracked right now
<code>git add <file> / .</code>	Moves changes from the working directory into the staging area
<code>git commit -m "msg"</code>	Saves a permanent, named snapshot of everything currently staged
<code>git log</code>	Shows the commit history, newest first
<code>git diff</code>	Shows the exact line-by-line changes that haven't been staged yet

git log — reading history

Raw `git log` can be overwhelming on a busy project, so most developers reach for a handful of flags to shape the view.

```
git log --oneline
git log --graph --all --decorate
git log -p
# show diffs per commit
git log --author="Sam"
git log --since="2 weeks ago"
```

git diff — inspecting changes

Different flags compare different pairs of snapshots — knowing which one you're looking at prevents a lot of confusion.

```
git diff                # unstaged vs
                        working dir
git diff --staged
# staged vs last commit
git diff HEAD~1 HEAD  # compare two
                        commits
```

Undo Operations (the safe, everyday kind)

Mistakes are constant in day-to-day coding — Git provides layered tools for undoing them, depending on exactly how far the change has gotten.

- `git restore <file>` — discard uncommitted changes in the working directory, reverting the file back to its last committed state.
- `git restore --staged <file>` — unstage a file without losing the actual edits; it goes back to being an unstaged change.
- `git checkout -- <file>` — the older, pre-2.23 equivalent of restore; still extremely common in tutorials and existing scripts.
- `git commit --amend` — fix the message or contents of your *most recent* commit only, without creating a new one.

Pro Tip: These are "local cleanup" tools, safe to use freely before anything is pushed. For undoing something already pushed and shared with teammates, jump to Chapter 5 (revert/reset) — the rules change once history is shared.

Chapter 3 — Branching & Merging

Branches are what make Git's workflow safe and parallel — isolate work, then bring it back together.

What is a Branch?

A branch is simply a lightweight, movable pointer to a commit — nothing more. Creating a new branch doesn't copy any files or duplicate the project; it just adds a new named pointer at your current commit. This is exactly why branching in Git is nearly instant and effectively free, unlike heavier, file-copying branch models in older version control tools.

As you keep committing on a branch, its pointer automatically moves forward to the newest commit. Multiple branches can point at different commits (or even the same one) at any time, which is what allows several independent lines of work to exist simultaneously in one repository.

Real-World Use: At most companies, main/production stays protected while every developer works in their own isolated feature branch, so a mistake in one person's unfinished work can never accidentally break the live application.

Creating & Switching Branches

```
git branch feature/login      # create a branch (stays on current branch)
git checkout feature/login    # switch to it
git checkout -b feature/login  # create + switch in one step
git switch -c feature/login    # modern alternative (Git 2.23+)
git branch                    # list local branches
git branch -d feature/login    # delete a branch already merged in
git branch -D feature/login    # force-delete, even if unmerged
```

Pro Tip: Use descriptive, prefixed names like `feature/login-page` or `fix/payment-bug` — it doubles as documentation and makes long branch lists easy to scan.

Merging — Fast-Forward vs Three-Way

When you run `git merge`, Git chooses one of two strategies depending on how far the branches have diverged.

Fast-Forward Merge	Three-Way Merge
Happens when the target branch hasn't moved since the feature branch was created — there's nothing to reconcile	Happens when both branches have new commits since they diverged
Git simply moves the target branch pointer forward to match — no new commit is created	Git creates a new "merge commit" with two parents, combining both histories
Produces a perfectly linear history, as if the work happened in sequence	Preserves the fact that two separate lines of work happened in parallel and were later joined

```
git checkout main
git merge feature/login
```

Merge Conflicts — Step by Step

A conflict happens when Git can't automatically decide how to combine two changes to the same lines — this requires a human decision, which is exactly what the manual resolution process is for.

- Git pauses the merge mid-way and marks every conflicting file directly inside the file content.
- Open the file and find the conflict markers: `<<<<<<` (your version begins), `=====` (divider), `>>>>>>` (their version ends).
- Manually edit the file to keep the correct version, a combination of both, or something new entirely, then delete all three marker lines.
- Stage the resolved file and finish the merge: `git add <file>` then `git commit`.

This is one of the most commonly whiteboarded practical questions in interviews — interviewers specifically want to hear the exact steps, not just a definition of what a conflict is.

Chapter 4 — Remote Repositories

Local Git becomes a team tool the moment you connect it to a remote host like GitHub.

GitHub Account & Creating a Repository

A GitHub repository is simply a cloud-hosted copy of a Git project, reachable over the network so multiple people (or machines, like CI runners) can read and write to it. There are two natural starting points: create an empty repository on GitHub first and clone it locally, or start locally with `git init` and connect it to GitHub afterward.

```
git clone https://github.com/user/repo.git # download an existing repo
git remote add origin https://github.com/user/repo.git # connect a local repo
git push -u origin main # first push, sets tracking
```

Push, Pull & Fetch

These three commands are how your local repository and the remote repository stay in sync — and mixing them up is one of the most common early points of confusion.

Command	What it does
<code>git push origin main</code>	Uploads your local commits to the remote branch
<code>git fetch</code>	Downloads new commits from the remote into your local copy, but does not merge or touch your working files
<code>git pull</code>	Runs <code>fetch</code> and then immediately <code>merge</code> s the remote changes into your current branch

Pro Tip: Senior developers often `fetch` first and review incoming changes with `git diff main origin/main` before merging, avoiding surprise conflicts landing unexpectedly.

Remote & Upstream

A **remote** is just a named reference to a repository hosted elsewhere — `origin` is the conventional default name Git assigns to whatever repository you cloned from. When you're working from a fork, it's standard practice to also add the original project as a second remote, conventionally named **upstream**, so you can pull in its latest changes.

```
git remote -v          # list remotes with their URLs
git remote add upstream <original-url> # add the original repo as upstream
git fetch upstream
git merge upstream/main
```

Real-World Use: Open-source contributors keep their fork synced with upstream regularly, so their pull requests are based on the latest code instead of arriving stale and full of avoidable conflicts.

Chapter 5 — Intermediate Git

The tools that separate "I know git add/commit/push" from someone who's genuinely comfortable working in real repositories.

Stash — Save Work Without Committing

Stash temporarily shelves all of your uncommitted changes — both staged and unstaged — and returns your working directory to a clean state matching the last commit. This lets you switch context immediately (change branches, pull updates, test something) without committing half-finished work just to get it out of the way.

```
git stash                # save tracked changes
git stash -u             # also stash untracked (new) files
git stash save "WIP: login" # label it for easy identification
git stash list           # see every stash you've saved
git stash pop            # reapply the latest stash and remove it
git stash apply          # reapply but keep it in the stash list too
```

Real-World Use: An urgent production bug lands while you're mid-feature — stash your half-done work, switch to a hotfix branch, ship the fix, then pop the stash to resume exactly where you left off.

Tags

A tag marks one specific commit as historically significant — almost always used to mark release versions, so anyone can check out exactly what code shipped at a given point.

```
git tag v1.0.0           # lightweight tag (just a name)
git tag -a v1.0.0 -m "Release 1.0" # annotated tag (preferred — stores message, author, date)
git push origin v1.0.0    # tags don't push automatically
git push origin --tags    # push all local tags at once
```

Pro Tip: Prefer annotated tags for anything user-facing like a release — they carry metadata that lightweight tags simply don't store.

Cherry-Pick

Cherry-pick applies one specific commit from any branch onto your current branch, without merging the entire branch it came from. Git replays just that one change as a brand-new commit on top of wherever you currently are.

```
git cherry-pick <commit-hash>
```

Real-World Use: A critical fix lands on `develop` but needs to go live on a production `hotfix` branch immediately — cherry-pick brings over just that one commit without dragging along everything else on develop.

Revert vs Reset vs Reflog

These three commands are all about undoing history, but they behave very differently — mixing them up on a shared branch is one of the most damaging mistakes a beginner can make.

Command	Behavior
<code>git revert <commit></code>	Creates a brand-new commit that reverses the changes — history is never rewritten, making it safe for shared/pushed branches
<code>git reset <commit></code>	Moves the branch pointer (and optionally the staging area/working directory) backward, effectively rewriting history — local use only , before pushing
<code>git reflog</code>	Shows a private log of every place HEAD has pointed, even commits no longer reachable from any branch — this is your safety net for recovering "lost" work

Never `reset` commits that have already been pushed and pulled by teammates — it breaks their local history the next time they try to sync. Use `revert` on anything shared.

Amending Commits

Sometimes you commit and immediately notice a typo in the message, or forget to include a file — `--amend` fixes the *most recent* commit in place instead of creating a whole new one.

```
git commit --amend -m "corrected message"
git commit --amend --no-edit # keep the same message, just add forgotten changes
```

Pro Tip: Only amend commits that haven't been pushed yet — amending rewrites the commit's hash, which causes the same shared-history problems as reset.

Chapter 6 — Advanced Git

Tools for cleaning up history, debugging regressions efficiently, and handling repositories at real-world scale.

Interactive Rebase & Squashing Commits

Rebase takes a sequence of commits and replays them onto a new base commit, one at a time — producing clean, linear history instead of the branching merge-commit shape a regular merge creates. Interactive mode opens an editable list of those commits, letting you reorder, edit, drop, or squash them before they land anywhere permanent.

```
git rebase -i HEAD~5 # interactively edit the last 5 commits
# in the editor: change "pick" to "squash" (or "s") to combine a commit into the one above
it
.
```

Never rebase commits that have already been pushed to a shared branch — since rebase rewrites commit hashes, it silently breaks history for anyone who already pulled the old version.

Real-World Use: Before opening a PR with 15 messy "wip" and "fix typo" commits, developers squash them into one clean, well-described commit so main's history stays readable for everyone who looks at it later.

Bisect — Binary-Search Debugging

`git bisect` performs an automated binary search through your commit history to pinpoint exactly which commit introduced a bug — dramatically faster than checking commits one at a time, since each test roughly halves the remaining search space.

```
git bisect start
git bisect bad           # the current commit is broken
git bisect good v1.0.0   # this old commit was known-good
# Git checks out a midpoint commit automatically – test it, then mark it:
git bisect good  # or
git bisect bad
# repeat until Git identifies the exact culprit commit
git bisect reset        # return to your original branch when done
```

Real-World Use: Instead of manually checking through 50 candidate commits, bisect narrows it down in roughly 6 steps ($\log_2 50 \approx 5.6$) — a huge time saver on a large, unfamiliar history.

Git Hooks

Hooks are scripts that Git runs automatically at specific points in its workflow — they live as executable files inside `.git/hooks/` and can be written in any scripting language.

- **pre-commit** — runs right before a commit is created; commonly used for linting or auto-formatting code
- **pre-push** — runs before pushing to a remote; a good place to run a fast test suite as a final safety check
- **commit-msg** — validates that commit messages follow a required format (e.g. Conventional Commits)

Pro Tip: Since hooks live locally inside `.git` and aren't tracked by Git itself, tools like Husky are popular for JavaScript projects — they install and share hooks automatically across a whole team.

Chapter 6 (continued) — Scaling Git

Worktree

`git worktree` checks out multiple branches of the same repository into separate folders simultaneously, sharing the same underlying `.git` history — no stashing or branch-switching required.

```
git worktree add ../hotfix hotfix-branch
```

Review a teammate's PR in one folder while your own unfinished feature branch stays checked out untouched in another.

Git LFS (Large File Storage)

Git wasn't designed for large binary files — every version of a large file gets stored in full, bloating the repository fast. LFS replaces those files with small text pointers inside Git, while the real binary content lives on a separate LFS server.

```
git lfs install
git lfs track "*.psd"
```

Game studios and ML teams use LFS to version large assets or datasets without bloating the core repository to gigabytes.

Submodules

A submodule embeds another Git repository as a subdirectory inside your main repository, pinned to one specific commit rather than tracking that project's latest changes automatically.

```
git submodule add <url> libs/shared
git submodule update --init --recursive
```

Many teams now prefer monorepos over submodules to avoid the extra operational complexity submodules introduce.

Sparse Checkout

Lets you check out only a subset of files from a large repository instead of the entire tree — useful when a monorepo is far too large to reasonably download in full for every contributor.

```
git sparse-checkout init --cone
git sparse-checkout set apps/web
```

Extra Cleanup & Inspection Tools

- `git clean -n` → preview which untracked files would be deleted (always run this first, before `-f`)
- `git clean -fd` → actually delete untracked files *and* directories
- `git show <commit>` → inspect exactly what changed in one specific commit, including the diff, author, and timestamp
- `git blame <file>` → see who last modified each line, and in which commit — the fastest way to trace where a bug was introduced

Chapter 7 — Git Internals

Understanding why Git behaves the way it does — this is where "I use Git" turns into "I understand Git."

The Git Object Model

Underneath every porcelain command you run, Git stores absolutely everything as one of four object types inside `.git/objects/`. Each object is identified purely by a hash of its own content — there's no separate "filename" tracked at this level at all.

Object	Represents
Blob	The raw content of a single file's contents — no filename, permissions, or metadata, just the data itself
Tree	A directory listing that maps filenames and permissions to blob or tree hashes, forming the folder structure
Commit	A snapshot pointer: references exactly one tree (the project state), one or more parent commits, plus author, committer, timestamp, and message
Tag	An annotated, named pointer to a commit, typically used to mark releases

Because a commit points to a tree, and a tree points to blobs (and possibly other trees for subdirectories), a single commit hash is enough to reconstruct the *entire* project state at that point in time.

SHA-1 / SHA-256 Hashing

Every object in Git is content-addressed: its ID is computed as a cryptographic hash of its own contents (plus a small header). Change even a single byte inside a file, and the resulting blob gets a completely different hash — which cascades upward, changing the tree's hash, and then the commit's hash too.

This is what gives Git its strong data integrity guarantee: two objects with the same hash are guaranteed (for all practical purposes) to have identical content, and any corruption or tampering is immediately detectable. Git historically used SHA-1 for this and is gradually adding support for the stronger SHA-256 algorithm.

References & HEAD

A **branch** is, at the filesystem level, nothing more than a small text file containing a single commit hash, stored under `.git/refs/heads/`. When you commit, Git updates that file to point at the new commit — this is the entire mechanism behind "a branch moving forward."

HEAD is a special reference that points to whichever branch (or, less commonly, a specific commit) you currently have checked out. Most of the time HEAD points at a branch name, which in turn points at a commit — this indirection is what lets "committing" automatically move the right branch forward.

Detached HEAD state: checking out a specific commit hash (instead of a branch name) points HEAD directly at that commit, bypassing any branch. Any new commits made there aren't attached to a branch and can become unreachable and effectively lost — unless you create a branch immediately with `git checkout -b new-branch` before doing anything else.

Packfiles

Freshly created objects start out as individual "loose" files, one per object. Over time, Git automatically compresses large numbers of these loose objects into a single, highly compressed **packfile**, storing deltas between similar objects (e.g. two nearly-identical versions of the same file) rather than repeating full copies of each one.

This packing process is exactly what keeps a repository with years of history — and thousands of nearly-identical commits — from becoming enormous on disk, and it's triggered automatically by commands like `git gc`.

Plumbing vs Porcelain Commands

Git's own documentation draws this distinction explicitly. **Porcelain** commands (`add` , `commit` , `log` , `merge`) are the polished, user-friendly interface most people use every day. **Plumbing** commands are the low-level building blocks underneath them, exposing the object model directly.

```
git hash-object -w file.txt    # store a file's content as a blob, and print its SHA
git cat-file -p <sha>         # inspect any object's raw, decompressed content
git rev-parse HEAD            # resolve HEAD down to its literal commit hash
```

You'll rarely need plumbing commands day-to-day, but they're exactly what interviewers use to probe for real understanding versus memorized porcelain commands — being able to explain what `git commit` does in terms of blobs, trees, and refs is a strong signal.

Chapter 8 — GitHub Collaboration Features

This is what turns a hosted Git repository into a full team platform — the layer built entirely on top of Git itself.

Issues

Issues track bugs, feature requests, and general tasks in one place, tied directly to the repository. Each can be labeled by type or priority, assigned to a specific person, and linked to the pull request that fixes it.

Use a keyword like `Fixes #12` in a PR description to auto-close that linked issue the moment the PR merges.

Pull Requests

A pull request proposes merging changes from one branch into another, and opens them up for structured review — comments, suggested edits, and approvals — before anything reaches the target branch.

At almost every professional team, no code reaches production without going through a PR first.

Fork

Forking creates a full personal copy of someone else's repository under your own account — the standard way to contribute to a project you don't have direct write access to, since you can freely make changes on your fork and later propose them back via PR.

Discussions

A forum-style space for open-ended Q&A, ideas, and announcements — intentionally separate from Issues, which are meant to represent concrete, actionable work items rather than open conversation.

Releases

A GitHub Release packages one specific tagged version of the code together with human-readable changelog notes and, optionally, downloadable build artifacts. Following semantic versioning (`v1.2.0` = `major.minor.patch`) makes releases predictable for anyone consuming them.

Projects

Kanban-style boards (commonly To Do → In Progress → Review → Done) that organize Issues and PRs visually. Many teams mirror their entire sprint board directly here instead of maintaining it in a separate external tool.

Wikis

A built-in, version-controlled documentation space attached to each repository — useful for onboarding docs, architecture notes, or anything that doesn't belong directly in the codebase.

GitHub Pages

Free static site hosting published directly from a repository, with zero separate hosting cost — commonly used for project documentation, a portfolio site, or a frontend demo, built automatically from a `docs/` folder or a dedicated `gh-pages` branch.

Chapter 9 — GitHub Actions (CI/CD)

Automating tests, builds, and deploys directly from your repository, triggered by the events that happen inside it.

Workflow, Jobs, Steps & Runners

GitHub Actions is organized as a small hierarchy of concepts, each nested inside the next.

- **Workflow** — an automated process defined in a YAML file under `.github/workflows/`, triggered by events like a push or a pull request.
- **Job** — a group of steps that all run together on the same runner. A workflow can contain multiple jobs, which run in parallel by default.
- **Step** — one individual command or reusable "action" inside a job, executed in order.
- **Runner** — the actual virtual machine (GitHub-hosted, or self-hosted for more control) that a job executes on.

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

Matrix Builds

A matrix build runs the exact same job repeatedly across multiple combinations of variables — different language versions, operating systems, or configuration flags — all in parallel, rather than writing a separate job for each combination by hand.

```
strategy:
  matrix:
    node-version: [18, 20, 22]
    os: [ubuntu-latest, windows-latest]
```

Secrets

Secrets are encrypted environment variables stored at the repository or organization level, injected into a workflow only at runtime. This is the correct way to hand a workflow access to API keys, database credentials, or deploy tokens — never hardcode them directly in workflow YAML, where they'd be visible to anyone reading the file.

```
env:  
  API_KEY: ${ secrets.MY_API_KEY }
```

Reusable Workflows

Define a workflow once in a central location, then call it from other workflows with `uses: org/repo/.github/workflows/build.yml@main` — this keeps large organizations from duplicating the same CI logic across dozens of repositories.

Deployments

A typical production pipeline runs tests and linting automatically on every pull request, then triggers a deployment to staging (or production) the moment code merges into main — turning "merge" into "ship" with no manual step in between.

Chapter 10 — GitHub Security

Built-in tooling that catches vulnerabilities and enforces safe collaboration before they turn into real incidents.

CodeQL

GitHub's semantic code analysis engine treats source code like queryable data, scanning it for known security vulnerability patterns and coding errors automatically as part of CI, catching issues before they ever reach a reviewer's eyes.

Dependabot

Continuously scans your project's dependencies against known vulnerability databases, then automatically opens pull requests to upgrade any affected package to a patched, safe version — turning a manual, easily-forgotten chore into an automated one.

Secret Scanning

Detects accidentally committed credentials — API keys, tokens, private keys — by pattern-matching known credential formats, and for many providers can block the push entirely before the secret ever lands in the repository's history.

Branch Protection

Enforces a set of rules directly on a branch: requiring pull request reviews before merge, requiring status checks (like CI) to pass, and blocking force-pushes that could otherwise silently rewrite shared history.

Rulesets

A newer, more flexible evolution of classic branch protection — rulesets let you apply the same kind of protection rules across multiple branches or tags at once, at either the repository or the entire organization level, instead of configuring each branch individually.

Real-World Use: Standard practice for a production repo is to require at least one approval, passing CI checks, and to block force-pushes to main — protecting the team from both bad code and accidental history rewrites.

Chapter 11 — GitHub API & Automation

Scripting GitHub itself — useful for internal tooling, bots, dashboards, and anything that needs to interact with repos programmatically.

REST API Basics

GitHub exposes nearly every feature of the platform — repositories, issues, pull requests, actions, users — over a REST API hosted at `api.github.com`. Requests are authenticated with either a personal access token or a GitHub App installation token, passed in the request headers.

```
curl -H "Authorization: token <TOKEN>" \  
https://api.github.com/repos/owner/repo/issues
```

GraphQL Overview

GitHub's GraphQL API lets you request exactly the fields you need — across multiple related resources — in a single request, avoiding the multiple sequential round-trips that REST sometimes requires when fetching deeply related data (e.g. a PR, its reviews, and its linked issues all at once).

GitHub CLI (gh)

The official command-line tool that wraps the API for everyday tasks, letting you manage issues, PRs, and repos entirely from the terminal without switching to a browser.

```
gh repo clone owner/repo
gh pr create --title "Fix bug" --body "Details..."
gh issue list
gh pr merge 42 --squash
```

Automation Examples

- Auto-label new issues based on keywords in the title, combining a GitHub Action trigger with a REST API call
- A review bot that comments on every PR automatically with test-coverage results
- A script that posts a message to a Slack channel via webhook every time a new release is published

Chapter 12 — Real-World Workflows

How individuals and teams actually organize branches day to day — pick the one that fits your release cadence and team size.

Solo Developer Workflow

Usually just `main` plus a handful of short-lived feature branches, committing directly and often. There's no review step, so simplicity wins over process — the priority is staying out of your own way.

Team Workflow

`main` (and often `develop`) stay protected; every change goes through a dedicated feature branch, a pull request, at least one review, and passing CI checks before it's allowed to merge.

Git Flow

A heavier, highly structured branching model built around dedicated long-lived branches: `main` (production), `develop` (integration), plus short-lived `feature/*`, `release/*`, and `hotfix/*` branches for their specific purposes.



Git Flow suits products with scheduled, versioned releases (e.g. mobile apps with app-store review cycles), where "what's in the next release" needs to be clearly separated from "what's still being integrated."

GitHub Flow

A much simpler model built around one principle: `main` is always deployable. Every change is a short-lived feature branch, merged via PR after review, and deployed right away — there's no separate `develop` branch to keep in sync.

Trunk-Based Development

Everyone commits small, frequent changes directly into a single shared trunk (or via extremely short-lived branches lasting hours, not days), often gating unfinished features behind feature flags rather than long-lived branches. This is the model favored by teams practicing true continuous deployment.

Pro Tip: Fast-moving startups and SaaS teams usually lean toward trunk-based development or GitHub Flow; larger, regulated, or release-scheduled products often lean toward Git Flow instead.

Chapter 13 — Best Practices

The habits that separate clean, maintainable repositories from ones nobody wants to touch six months later.

Writing Good Commit Messages

- Use the imperative mood: "Fix login bug" rather than "Fixed" or "Fixes" — this matches how Git's own auto-generated messages read.
- Keep the summary line under roughly 50 characters, and add further detail in the body if needed.
- Explain *why* the change was made, not just *what* changed — the diff itself already shows the "what."
- Reference related issue numbers where relevant, e.g. "Fix #42: handle null token", so history stays traceable back to the reasoning.

Branch Naming Conventions

- feature/short-description
- fix/bug-description
- hotfix/critical-issue
- chore/dependency-update

A consistent prefix convention makes long branch lists instantly scannable, and lets tooling (like auto-labeling bots) key off the prefix automatically.

Keeping History Clean

- Squash noisy WIP commits into one meaningful commit before merging
- Rebasing local feature branches onto the latest main before opening a PR, so the diff is easy to review
- Never rewrite (rebase/reset/amend) history that's already been shared with others

Resolving Conflicts Efficiently

- Pull or rebase onto main frequently — small, frequent conflicts are far easier to resolve than one giant one at the end
- Communicate proactively with teammates who are editing the same files
- Use a visual diff/merge tool for anything beyond a trivial one-line conflict

Repository Organization

- Keep a clear README, CONTRIBUTING.md, and LICENSE at the repository root
- Use a CODEOWNERS file to auto-assign the right reviewers by area of the codebase
- Maintain a consistent folder structure across services or packages in a monorepo, so navigation stays predictable

Chapter 14 — Hands-On, Mistakes & Cheat Sheet

Reading isn't enough — Git only really clicks once you've broken something on purpose and fixed it yourself.

Practice Tasks (do these in order)

- **Ch.2** — Init a repo, make three commits, and inspect the history with `git log --graph .`
- **Ch.3** — Create two branches that deliberately edit the same line of a file, merge them, and resolve the resulting conflict by hand, start to finish.
- **Ch.4** — Push a repo to GitHub, clone it into a second folder to simulate a teammate, make changes in both, and practice the difference between fetch and pull.
- **Ch.5** — Stash a change mid-edit, switch branches, then pop it back. Separately, create and push an annotated tag.
- **Ch.6** — Make five intentionally messy commits and squash them with interactive rebase; then use bisect on a small repo with a known bad commit buried in its history.
- **Ch.7** — Run `git cat-file -p HEAD` and manually trace the commit → tree → blob chain for a small file.
- **Ch.9** — Write a GitHub Actions workflow from scratch that runs your test suite automatically on every push.

Mini Projects

- **Personal changelog site** — deploy a simple static page via GitHub Pages, and tag every meaningful update as a proper release.
- **Open-source first contribution** — fork a small, welcoming repo, fix a "good first issue," and open a PR that follows its CONTRIBUTING.md guidelines exactly.
- **CI pipeline from scratch** — add a lint-and-test GitHub Actions workflow to any existing project, then require it to pass via branch protection before merges are allowed.
- **Team simulation** — with a friend, both clone the same repo and deliberately create a merge conflict together, then practice talking through and resolving it as if it were a real team situation.

Common Mistakes to Avoid

- Committing `node_modules/` , `.env` , or build output because `.gitignore` wasn't set up before the first commit
- Using `git reset --hard` on shared history instead of the safer `git revert`
- Force-pushing directly to main without any branch protection in place
- Writing vague, unhelpful commit messages like "fix stuff" or "update"
- Working directly on main instead of isolating changes in a feature branch
- Forgetting to pull before starting new work, leading to entirely avoidable conflicts later
- Committing secrets or API keys directly into the repository, even temporarily

Interview Quick-Fire — Question Bank by Level

Use these as self-test prompts. For full worked answers with pro tips and real-world context, see the companion **Git & GitHub Interview Questions** notes (52 Q&A).

Beginner

- What is Git, and why is it "distributed"?
- Difference between Git and GitHub?
- What do staging area and working directory mean?
- How do you create and switch to a new branch?
- What does .gitignore do, and why does order of operations matter?

Intermediate

- git merge vs git rebase — tradeoffs?
- Walk through resolving a merge conflict step by step.
- git reset vs git revert — when would you use each?
- What's the difference between git fetch and git pull?
- When would you use git stash vs committing WIP work?

Advanced

- Explain Git's object model — blob, tree, commit, tag.
- How does git bisect work internally?
- What happens during a fast-forward vs three-way merge at the ref level?
- How would you recover a branch you accidentally deleted?
- Explain porcelain vs plumbing commands with examples.

Scenario-Based

- You pushed a commit with a secret key in it. What do you do?
- A teammate's branch has 20 noisy commits — how do you clean it up before merging?
- Production is broken and the last 3 commits are suspects — how do you find the culprit fast?
- Two developers edited the same function differently and both pushed — walk through the fix.

Full Command Cheat Sheet

Setup & Basics

```
git init
git clone <url>
git config --global user.name "...
git status
git add . / <file>
git commit -m "msg"
git commit -am "msg"
```

History

```
git log --oneline --graph --all
git diff / --staged
git show <commit>
git blame <file>
```

Branching & Merging

```
git branch / -d <name>
git checkout -b <name>
git switch -c <name>
git merge <branch>
git rebase <branch>
git rebase -i HEAD~n
```

Remote

```
git remote add origin <url>
git push origin <branch>
git fetch
git pull
git pull --rebase
```

Undo & Recover

```
git restore <file>
git restore --staged <file>
git commit --amend
git revert <commit>
git reset --soft/--hard <commit>
git reflog
```

Advanced

```
git stash / stash pop
git cherry-pick <commit>
git tag -a v1.0 -m "msg"
git bisect start/good/bad
git worktree add ../dir branch
git submodule update --init
```

Print this page, pin it near your desk, and you'll rarely need to search "how to undo git commit" again.